


```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024

int main()
{
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    int fd;
    pid_t pid;

    printf("=====\n");
    printf("      INPUT REDIRECTION\n");
    printf("=====\n");

    printf("Enter input filename: ");

    if (fgets(filename, sizeof(filename), stdin) == NULL)
        return 1;

    filename[strlen(filename)] = '\0';

    fd = open(filename, O_RDONLY);

    if (fd == -1)
    {
        perror("Unable to open input file");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }
}
```

```
fd = open(filename, O_RDONLY);

if (fd == -1)
{
    perror("Unable to open input file");
    return 1;
}

pid = fork();

if (pid < 0)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    /*
     * Redirect standard input to file.
     */
    if (dup2(fd, STDIN_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);

    /*
     * Execute cat.
     * cat reads from redirected stdin.
     */
    execlp("cat", "cat", NULL);

    perror("execlp");
    exit(EXIT_FAILURE);
}

/*
 * Parent keeps its original stdin.
 */
close(fd);

waitpid(pid, NULL, 0);
```

```
manchodu@Srini:~/OSLAB$ nano m28.c
manchodu@Srini:~/OSLAB$ gcc m28.c -o m28
manchodu@Srini:~/OSLAB$ ./m28
```

```
=====
                OUTPUT REDIRECTION
=====
Enter output filename: m12

Output redirection completed.
Output written to: m12
manchodu@Srini:~/OSLAB$
```

```
manchodu@Srini:~/OSLAB$ cat m28.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024

int main()
{
    char filename[MAX_INPUT];

    int fd;
    pid_t pid;

    printf("=====\n");
    printf("                OUTPUT REDIRECTION\n");
    printf("=====\n");

    printf("Enter output filename: ");

    if (fgets(filename, sizeof(filename), stdin) == NULL)
        return 1;

    filename[strcspn(filename, "\n")] = '\0';

    /*
     * Create/open output file.
     *
     * O_WRONLY -> write only
     * O_CREAT  -> create if it does not exist
     * O_TRUNC  -> clear existing contents
     */
    fd = open(
        filename,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
```

```

        return 1;

filename[strlen(filename, "\n")] = '\0';

/*
 * Create/open output file.
 *
 * O_WRONLY -> write only
 * O_CREAT  -> create if it does not exist
 * O_TRUNC  -> clear existing contents
 */
fd = open(
    filename,
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (fd == -1)
{
    perror("Unable to create output file");
    return 1;
}

pid = fork();

if (pid < 0)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    /*
     * Redirect standard output to file.
     */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(EXIT_FAILURE);
    }
}

```

```
if (pid == 0)
{
    /*
     * Redirect standard output to file.
     */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);

    /*
     * Everything printed by this command
     * goes into the output file.
     */
    printf("Hello from Linux output redirection.\n");
    printf("This data is stored in the output file.\n");
    printf("stdout has been redirected successfully.\n");

    fflush(stdout);

    exit(EXIT_SUCCESS);
}

/*
 * Parent closes its copy of the descriptor.
 */
close(fd);

waitpid(pid, NULL, 0);

printf("\nOutput redirection completed.\n");
printf("Output written to: %s\n", filename);

return 0;
}
```